Database Architecture & ERD Design Report

Phase 1

Sports Event Ticket Booking & Reservation Platform

Technical Documentation | Relational Database Systems

Database Engine	PostgreSQL 16+ (UTF-8)	**Normalization Target**	Third Normal Form (3NF)
Platform Type	High-Concurrency Ticketing System	**Architecture Pattern**	Vertical Table Extension

8	4	3	6
Core Tables	ENUM Domains	Sport Extensions	Optimized Indexes

Abstract

This document presents the Phase 1 relational database design for a high-concurrency sports event ticketing and reservation platform. It formalizes eight domain-partitioned PostgreSQL tables — covering user accounts, generalized ticket listings, sport-specific seating extensions for football, volleyball, and basketball, temporary reservation holds, financial payment records, and customer support reports — all normalized to Third Normal Form (3NF). The report documents the entity design rationale, declarative integrity constraints (CHECK, ENUM, and cascading/restricting foreign keys), and the indexing strategy adopted to sustain fast lookups and full-text-style search under concurrent load.

Contents

1. Introduction & Architectural Objectives

The primary objective of Phase 1 is to perform a domain analysis for a high-traffic sports ticketing platform and construct an optimal, fully normalized relational schema. The architecture eliminates data redundancies and update anomalies by strictly adhering to Third Normal Form (3NF). It establishes a robust foundation for high-throughput transactional processing (ACID) and efficient query execution under heavy concurrent traffic.

2. Database Entities & System Schema Overview

The system architecture comprises **8 interconnected, domain-partitioned tables**:

1. `users` **Table:** Stores user demographics, hashed credentials (`password_hash`), role-based access (`role`), and account status flags (`is_active`). To satisfy 1NF atomicity, names are decomposed into `first_name` and `last_name`.
2. `tickets` **Table:** Generalized base entity storing shared match attributes including `home_team`, `away_team`, `sport_type`, `ticket_tier`, `organizer`, `venue_name`, `city`, `match_date`, `price`, and `remaining_capacity`.
3. **Sport Extension Tables (`football_details`, `volleyball_details`, `basketball_details`):** Specialized 1:1 child entities isolating discipline-specific metadata (seating layout, tier, amenities) without polluting the base `tickets` table with excessive NULL values.
4. `reservations` **Table:** Controls temporary ticket holds with expiration timeouts (`expires_at`) prior to payment, and tracks lifecycle `status` (`pending`/`paid`/`cancelled`). A nullable `cancelled_by_support_id` foreign key records which support agent (a `users` row with role `support`) cancelled the reservation, when applicable.
5. `payments` **Table:** Captures transaction ledgers linked via a 1:1 relation with `reservations` for financial integrity. Its foreign keys use `ON DELETE RESTRICT` rather than `CASCADE`, so a paid reservation or the paying user can never be deleted while a payment record still references them.
6. `reports` **Table:** Logs customer support issues using dual nullable foreign keys (`ticket_id` and `reservation_id`) so a complaint can reference either a ticket or a reservation independently.

3. Normalization Analysis & 3NF Justification

All 8 schema entities satisfy Third Normal Form (3NF) standards:

- **First Normal Form (1NF):** Attributes are strictly atomic (`first_name`, `last_name`, `home_team`, `away_team`). Multi-valued attributes and repeating groups are converted into normalized child extension tables.
- **Second Normal Form (2NF):** Fulfills 1NF. Non-prime attributes are fully functionally dependent on the primary key (PK). Extension tables reuse `ticket_id` as both primary key and foreign key, eliminating partial dependencies on a composite key.
- **Third Normal Form (3NF):** Contains zero transitive functional dependencies ($X \rightarrow Y \rightarrow Z$). Specialized sport metadata is vertically partitioned into dedicated 1:1 extension tables, eliminating sparse, NULL-heavy base tables.

4. Schema Architecture Summary

The following table provides a comprehensive architectural summary of all system entities, primary key identifiers, foreign key dependencies, and their underlying 3NF normalization rationales:

Table Name	Primary Key	Foreign Key(s)	3NF Design Rationale & Architectural Purpose
users	user_id	None	Enforces unique email/phone constraints; atomic name fields; decouples authentication credentials from application data.
tickets	ticket_id	None	Generalized match metadata shared across all sports; decouples team names into home_team and away_team.
football_details	ticket_id	ticket_id → tickets	1:1 extension isolating stadium name, stand section, row/seat positioning, ticket type, and outdoor amenities.
volleyball_details	ticket_id	ticket_id → tickets	1:1 extension isolating indoor hall layout, seating section, row/seat allocation, ticket tier, and hall amenities.
basketball_details	ticket_id	ticket_id → tickets	1:1 extension isolating arena layout, seating section, row/seat allocation, ticket tier, and courtside amenities.
reservations	reservation_id	user_id → users ticket_id → tickets cancelled_by_support_id → users (nullable)	Manages transient ticket-hold lifecycles and expiration logic without redundant match or spectator data storage. The nullable cancelled_by_support_id foreign key preserves an audit trail of which support agent cancelled a reservation.
payments	payment_id	reservation_id → reservations (unique) user_id → users	1:1 financial transaction ledger fully decoupled from transient reservation state to ensure auditability; ON DELETE RESTRICT on both keys protects completed transactions from accidental loss.
reports	report_id	user_id → users ticket_id → tickets (nullable) reservation_id → reservations (nullable)	Independent customer support logging; uses dual nullable foreign keys to reference either a ticket or a reservation.

5. Integrity Constraints & Domain Business Logic

To enforce data consistency at the database engine layer and prevent invalid state transitions, declarative constraints and PostgreSQL custom domain enumerations are defined:

Numeric & Value Invariants (CHECK)

- Non-negative ticket pricing: enforced via `CHECK (price >= 0)` to eliminate negative billing anomalies.

- Non-negative remaining capacity: enforced via `CHECK (remaining_capacity >= 0)` to guarantee seat allocation boundaries are never exceeded.
- Non-negative settlement amounts: enforced via `CHECK (amount >= 0)` in the financial transaction ledger.

Temporal Consistency & Expiration Logic

- Valid reservation timeframe: enforced via `CHECK (reserved_at < expires_at)` (`check_reservation_dates`), guaranteeing that expiration timeouts are strictly chronological and enabling automated cleanup routines.

Domain Enumerations (ENUM)

- User Roles (`user_role`): bounded domain restricted to `'audience'` and `'support'`.
- Reservation Lifecycle Status (`reservation_status`): domain constrained to `'pending'`, `'paid'`, and `'cancelled'`.
- Financial Payment Status (`payment_status`): domain restricted to `'successful'`, `'failed'`, and `'pending'`.
- Sport Category Enumeration (`sport_type_enum`): domain bounded to `'football'`, `'volleyball'`, and `'basketball'`.

Referential Integrity & Cascading Actions

- **ON DELETE CASCADE — Account & Ticket Cleanup:** automatically purges a user's associated reservations when the user profile is deleted, and purges a ticket's extension row and reservations when the ticket itself is deleted.
- **ON DELETE SET NULL — Audit Preservation:** preserves customer support logs when a referenced ticket or reservation is removed (`reports.ticket_id`, `reports.reservation_id`); likewise preserves a reservation's history when the cancelling support agent's account is removed (`reservations.cancelled_by_support_id`).
- **ON DELETE RESTRICT — Financial Integrity Lock:** blocks deletion of any reservation or user that already has a payment record (`payments.reservation_id`, `payments.user_id`), forcing explicit handling of financial history instead of silent, cascading loss of transaction data.

6. Performance Indexing & Query Optimization Strategy

To accelerate high-frequency queries, multi-attribute wildcard lookups, and concurrent event searches, a multi-tiered indexing strategy is implemented:

Compound B-Tree Indexes

- **Multi-Column Filter Index** (`idx_tickets_sport_city`): constructed on `(sport_type, city)` to optimize composite filtering during active ticket discovery.
- **Chronological Order Index** (`idx_tickets_match_date`): configured on `match_date ASC` to facilitate fast time-range scans and chronological match listings.
- **User Activity Index** (`idx_reservations_user_status`): constructed on `(user_id, status)` to accelerate lookups of a specific user's reservations filtered by lifecycle state, e.g. listing all pending or paid reservations for one account.

GIN Trigram Indexes (pg_trgm)

- The `pg_trgm` extension is enabled and GIN trigram indexes are configured on `venue_name`, `home_team`, and `away_team`.
- These accelerate sub-string wildcard searches (`ILIKE '%term%'`) across team names and venue titles, reducing lookup cost from full table scans to inverted-index scans.

Query Execution Plan Optimizations

- Primary Key B-Tree indexing is automatically generated on all surrogate primary keys (`user_id`, `ticket_id`, `reservation_id`, `payment_id`, `report_id`).
- Unique indexes are enforced on `phone_number`, `email`, and `payments.reservation_id` to guarantee fast constraint checks during user login and payment verification.

7. Entity-Relationship Data Dictionary & Structural Analysis

The complete structural breakdown of all system entities, primary attributes, foreign key relationships, and cardinalities is formally detailed below:

Entity / Table	Attributes & Keys	Relationships	Cardinality & Target
users	user_id (PK) first_name, last_name phone_number, email password_hash, role city, is_active, created_at	Makes Reservation Processes Payment Submits Support Report Cancels Reservation (as support)	1:N → reservations 1:N → payments 1:N → reports 1:N → reservations
tickets	ticket_id (PK) home_team, away_team sport_type, ticket_tier organizer, venue_name, city match_date, price remaining_capacity is_active, created_at	Booked In Reservation Has Football Details Has Volleyball Details Has Basketball Details Referenced By Support Report	1:N → reservations 1:1 → football_details 1:1 → volleyball_details 1:1 → basketball_details 1:N → reports
football_details	ticket_id (PK/FK) league_name, stadium_name stand_section row_number, seat_number ticket_type, amenities	Extension of Base Ticket Entity	1:1 → tickets
volleyball_details	ticket_id (PK/FK) league_name, hall_name seat_section row_number, seat_number ticket_tier, amenities	Extension of Base Ticket Entity	1:1 → tickets
basketball_details	ticket_id (PK/FK) league_name, hall_name seat_section row_number, seat_number ticket_tier, amenities	Extension of Base Ticket Entity	1:1 → tickets
reservations	reservation_id (PK) user_id (FK)	Booked By User Books Ticket	N:1 → users N:1 → tickets

	ticket_id (FK) status, reserved_at expires_at cancelled_by_support_id (FK, Nullable)	Settled By Payment Ledger Referenced By Support Report Cancelled By Support Agent	1:1 → payments 1:N → reports N:1 → users (nullable)
payments	payment_id (PK) reservation_id (FK, Unique) user_id (FK) amount, status payment_method, paid_at	Settles User Reservation	1:1 → reservations N:1 → users
reports	report_id (PK) user_id (FK) ticket_id (FK, Nullable) reservation_id (FK, Nullable) category, report_text status, created_at	Submitted By User Account Refers To Ticket Refers To Reservation	N:1 → users N:1 → tickets N:1 → reservations

8. Phase 1 Executive Summary & Architectural Conclusion

Phase 1 successfully delivers a fully normalized Third Normal Form (3NF) database architecture in PostgreSQL, engineered specifically for high-concurrency sports event ticketing and reservation workflows. Through an exhaustive domain analysis, all user demographic attributes, base ticket metadata, discipline-specific details, transient holds, financial transaction ledgers, and support ticket logs have been cleanly partitioned into 8 cohesive tables.

8.1 Key Architectural Milestones Achieved

- **Atomic & Normalized Attribute Decomposition:** user identities are split into first_name and last_name, while match titles are separated into home_team and away_team, eliminating 1NF non-atomic string structures and enabling indexed team filtering.
- **Extensible 1:1 Vertical Table Extensions:** discipline-specific features for football, volleyball, and basketball are isolated in child extension tables. This prevents base tickets table pollution from excessive NULL values while allowing seamless addition of future sport types.
- **Engine-Level Data Integrity:** strict declarative CHECK constraints, custom PostgreSQL ENUM domains, a temporal validity rule (reserved_at < expires_at), and a full range of foreign key actions — CASCADE, SET NULL, and RESTRICT — preserve transactional and financial consistency without relying solely on application-level validation.
- **High-Performance Indexing Strategy:** compound B-Tree indexes and GIN trigram indexes (pg_trgm) guarantee low query-time complexity for range filtering, chronological sorting, per-user reservation lookups, and fuzzy sub-string team searches under high concurrent load.

8.2 Production Readiness & Phase 2 Integration

The designed relational schema establishes a robust foundation for Phase 2 backend implementation. The structure is fully compatible with standard PostgreSQL Data Definition Language (DDL) scripts, stored procedures (PL/pgSQL), database triggers for capacity management, and downstream integration with application-layer web services and caching layers.